



Variáveis e Tipos

Aula 02 · Bloco 1 — Fundamentos

Tipagem estática, casting e a armadilha da divisão inteira

Nesta aula

1. **Tipagem estática** — declarar é assumir compromisso
2. Os tipos **primitivos**
3. `String` não é primitivo
4. **Casting** e a divisão inteira
5. `final` e o `Scanner`

Declarar é assumir compromisso

```
int idade = 19;           // esta caixa só aceita inteiros
String nome = "Maria";    // esta só aceita texto
double preco = 19.90;     // esta só aceita decimais

idade = 20;               // ✅ continua sendo int
idade = "vinte";          // ❌ incompatible types
```

O tipo é decidido na declaração e checado pelo compilador **antes** de o programa rodar.

Parece burocracia — e é. Em troca, uma classe inteira de bugs simplesmente **não acontece**.

Os quatro primitivos que resolvem o curso

```
int quantidade = 42;           // inteiros
double preco = 19.90;         // decimais
boolean aprovado = true;      // só true ou false
char inicial = 'M';           // UM caractere, ASPAS SIMPLES
```

São 8 no total; os outros quatro (`long` , `float` , `short` , `byte`) aparecem bem menos.

⚠ **`char` usa aspas simples; `String` usa aspas duplas.** `'M'` é um caractere, `"M"` é um texto de tamanho 1. Trocar dá erro de compilação.

⚠ Variável local não tem valor padrão

```
int soma;  
System.out.println(soma);
```

```
error: variable soma might not  
have been initialized
```

O compilador **não deixa** você usar
uma variável que ainda não recebeu valor.

String não é primitivo — é objeto

Começa com maiúscula porque é uma **classe**. Texto em Java vem com **métodos** de brinde:

```
String frase = "Java é divertido";

frase.length();           // 16
frase.toUpperCase();       // JAVA É DIVERTIDO
frase.contains("Java");    // true
frase.charAt(0);           // J – começa do zero!
frase.substring(0, 4);     // Java – do 0 até ANTES do 4
"  espaços ".trim();      // remove as bordas
```

Formatando a saída com `printf`

```
System.out.printf("Aluno: %s | Média: %.2f%n", nome, media);
```

`%s` texto

`%d` inteiro

`%.2f` decimal com 2 casas

`%n` quebra de linha

O `printf` usa a **configuração regional**: em português, o separador decimal é vírgula. Não é bug.

Casting: quando o compilador exige responsabilidade

```
int inteiro = 10;  
double comDecimais = inteiro;    // ✅ automático – int cabe folgado  
  
double valor = 9.87;  
int truncado = valor;            // ❌ possible lossy conversion  
int certo = (int) valor;         // ✅ 9 – você assume a responsabilidade
```

⚠️ O cast **trunca**, não arredonda. `(int) 9.87` é `9`, não `10`.

O erro que mais estraga média no mundo

```
int soma = 7, quantidade = 2;  
soma / quantidade      → 3      ← divisão INTEIRA  
(double) soma / quantidade → 3.5  ✓  
7 / 2.0                → 3.5  ✓
```

int / int sempre dá int .

Quer casas decimais? Pelo menos um lado precisa ser

double .

final — quando o valor não pode mudar

```
final double PI = 3.14159;  
final int IDADE_MINIMA = 18;  
  
PI = 3.15;      // ✗ cannot assign a value to final variable
```

Constantes vão em **MAIUSCULA_COM_UNDERLINE**, por convenção.

Use **final** sempre que um valor **não deveria** mudar — o compilador passa a proteger essa decisão por você.

Lendo do teclado: **Scanner**

```
import java.util.Scanner;           // antes da classe!

Scanner scanner = new Scanner(System.in);

String nome = scanner.nextLine();   // linha inteira, com espaços
int idade = scanner.nextInt();      // um inteiro
double altura = scanner.nextDouble(); // um decimal

scanner.close();                    // libera o recurso
```

💡 `nextDouble()` espera o separador do seu sistema: digite `1,75` , não `1.75` .

! A pegadinha clássica do **Scanner**

`nextInt()` lê o número —
mas **deixa o Enter para trás**.

O `nextLine()` seguinte engole esse Enter
e devolve string vazia. Parece que o programa "pulou" a
pergunta.

Solução: um `scanner.nextLine()` extra logo depois
do `nextInt()`.

Exercícios da aula

Na pasta `aula-02/` :

1. **Apresentacao.java** — `String` , `int` , `double` , `boolean` numa frase com `printf` ;
2. **Conversor.java** — dólar para real, e Celsius→Fahrenheit. **Por que** `9/5` dá 1?
3. **Previsao.java** — preveja num comentário antes de rodar: `7/2` , `7/2.0` , `(int) 7.9` , `10/4*4` ;
4. **Calculadora.java** — dois `double` do teclado, cinco operações com `%.2f` ;
5. **Desafio** 🌶️ **Imc.java** — e explique num comentário por que o `nextLine()` extra foi necessário.

Próxima aula

Aula 03 — Operadores e Condicionais

Decidir com `if` e `switch` —
e comparar textos do jeito certo.